

PetCare AI

Definição da Funcionalidade de Inteligência Artificial — Sprint 3

Disciplina: Disruptive Architectures • Challenge FIAP 2026 • Parceiro: CLYVO VET

1. O problema que a PetCare AI resolve

O PetCare Hub já coleta, de forma contínua, dados reais sobre a rotina do pet: movimento (coleira), alimentação (comedouro) e condições ambientais. Hoje, esses dados alimentam um **Score de Saúde** (0-100) e geram **alertas** quando algum indicador foge do padrão esperado.

O problema é que dado bruto e um número de score, sozinhos, não comunicam nada de útil pro tutor. Um score que caiu de 82 para 61 não diz *por que* caiu, nem *o que fazer* a respeito. A **PetCare AI** existe pra preencher exatamente essa lacuna: ela lê o contexto completo do pet e traduz isso em uma orientação simples, humana e acionável.

Frase-síntese: a IA interpreta sinais de mudança de comportamento e orienta o tutor sobre quando vale a pena buscar avaliação veterinária — ela nunca diagnostica, nunca substitui o profissional.

Esse limite (interpretar, não diagnosticar) é uma decisão de produto deliberada, e está refletida em todas as escolhas técnicas descritas neste documento.

2. Experiência do usuário

1. O tutor abre a tela “PetCare AI” no aplicativo.
2. O sistema já analisou automaticamente os dados recentes do pet (ou o tutor pode perguntar algo específico).
3. A IA responde com uma explicação em linguagem simples, um nível de urgência visual (verde = baixa / amarelo = média / vermelho = alta), e, quando aplicável, uma ação sugerida (ex: “observação por 24h”, “agendar checkup”).

Exemplo de saída real:

“Rex está bem mais parado que o normal e comeu menos hoje. Fique de olho em outros sinais, como vômito ou dificuldade para se locomover. Se o comportamento continuar até amanhã, vale procurar a clínica.”

Notificação ativa: o tutor não precisa lembrar de abrir o app. Quando a urgência calculada for **média** ou **alta**, o sistema dispara uma notificação push, além de deixar o card pronto na tela. Para urgência **baixa**, a informação fica disponível de forma passiva (só quando o tutor abrir o app), evitando notificação desnecessária.

3. Dados utilizados pela IA

Fonte	Dado	Papel na análise
Cadastro do Pet	Espécie, idade, raça, porte	Personaliza a análise e a busca de diretrizes
Leituras da coleira	Nível de atividade, variação de movimento	Detecta padrão de comportamento
Leituras do comedouro	Percentual de ração consumida	Detecta mudança de apetite
Score de Saúde	Score atual e histórico	Prioriza gravidade e tendência
Alertas	Alertas ativos e resolvidos	Explica o motivo por trás de um alerta
Histórico recente	Leituras dos últimos dias	Evita decisão baseada em um dado isolado
Consultas	Histórico de consultas, diagnósticos e retornos recomendados	Evita sugerir uma consulta nova se já existe uma marcada; contextualiza o padrão com o que já foi observado clinicamente
Eventos preventivos	Vacinas e cuidados programados (aplicados e pendentes)	Combina o alerta atual com pendências preventivas (ex: vacina atrasada), e evita interpretar como anômalo um efeito colateral esperado de um cuidado recente
Base de conhecimento veterinário	Diretrizes preventivas por espécie/idade/porte	Fundamenta a resposta em conhecimento técnico real, evitando alucinação

A IA **nunca** acessa o banco de dados diretamente — todo o contexto é montado e controlado pelo Java, que é o backend responsável por toda a regra de negócio do PetCare Hub. Consultas e eventos preventivos já existem como entidades no sistema (usadas por outras partes do PetCare Hub); a IA passa a também consultá-las como fonte de contexto adicional, sem exigir nenhuma mudança de banco ou de modelagem.

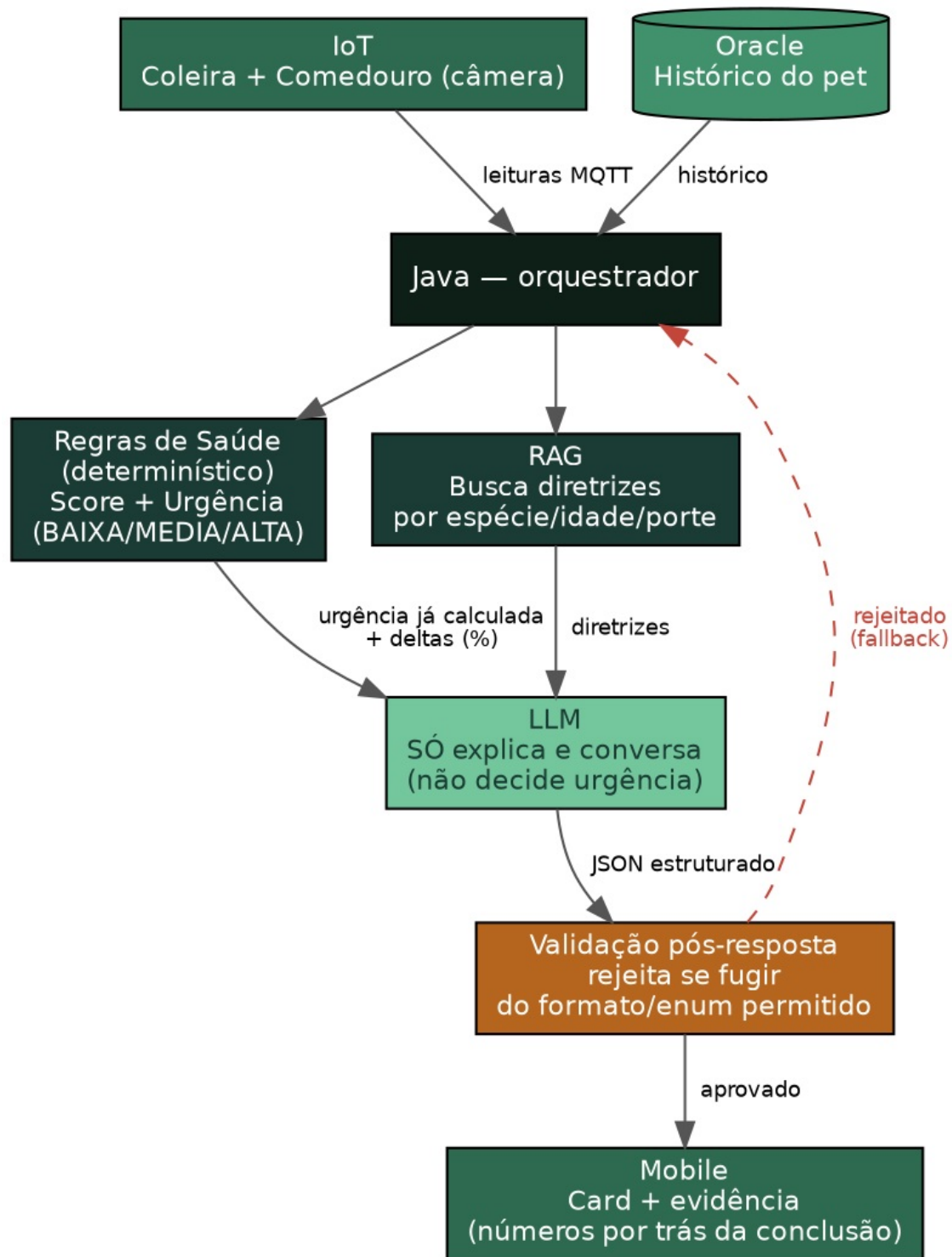
4. Estratégia de personalização

A PetCare AI não aplica o mesmo limiar ou o mesmo texto pra todos os pets — a personalização acontece em três camadas, todas já presentes no desenho das seções anteriores:

- **Perfil individual do pet:** espécie, idade e porte filtram quais diretrizes a camada de RAG (seção 5) recupera da base de conhecimento veterinário — um filhote de gato não recebe as mesmas referências de um cão adulto de porte grande.
- **Baseline próprio, não populacional:** as comparações mostradas ao tutor (“Atividade Hoje: 39, Média: 75”, seção 6) usam a média histórica **daquele pet específico**, não um valor fixo válido pra qualquer animal. Isso evita alarme falso pra pets naturalmente mais quietos ou mais agitados.
- **Contexto clínico do próprio pet:** o histórico de consultas e eventos preventivos (seção 3) evita repetir uma sugestão já tomada (“agendar checkup” quando já existe um marcado) e evita interpretar como anomalia algo que é efeito esperado de um cuidado recente daquele pet.

Essa personalização acontece toda na etapa determinística, **antes** da LLM entrar em cena (seção 5) — ou seja, o “perfil” do pet nunca depende de a IA generativa lembrar ou inferir nada sobre ele; é dado buscado e filtrado pelo Java a cada chamada.

5. Arquitetura



Arquitetura PetCare AI

Fluxo: 1. IoT (coleira e comedouro) envia leituras via MQTT continuamente; o Java persiste no Oracle. 2. **O Java calcula, de forma determinística, o que já pode ser calculado sem IA nenhuma:** a variação percentual de cada indicador (atividade, consumo) e o **nível de urgência**, a partir das faixas do Score de Saúde que já existem no sistema (80-100 = baixa, 50-79 = média, 0-49 = alta). 3. O Java consulta a base de conhecimento veterinário, filtrando por características do pet (espécie, faixa etária, porte) — essa é a camada de **RAG (Retrieval-Augmented Generation)**. 4. Os dados já calculados (incluindo a urgência) e as diretrizes recuperadas são enviados à LLM, que tem uma responsabilidade deliberadamente restrita: **explicar em linguagem humana** por que aquele padrão é relevante, e escolher uma ação sugerida dentro de uma lista fechada. A LLM nunca decide nem pode contradizer a urgência que o sistema já calculou. 5. A resposta passa por uma **validação**: se algum campo vier fora do formato ou da lista de valores permitidos, o Java corrige ou usa um valor seguro padrão antes de repassar ao Mobile. 6. O Mobile exibe o card com a explicação **e a evidência** — os números que embasam a conclusão (ex: “atividade

hoje: 39, média: 75, ↓48%”) — para que o tutor veja de onde a resposta veio, não apenas confie cegamente nela.

Por que a IA não decide a gravidade

A decisão mais sensível do produto — o quão urgente é a situação — é responsabilidade de uma regra determinística e auditável (o Score de Saúde), não da LLM. Isso reduz o risco de alucinação exatamente onde ele seria mais perigoso, e torna a arquitetura mais defensável: a IA generativa entra como camada de interpretação e comunicação, não como tomadora de decisão clínica. O formato estruturado, somado à validação pós-resposta, garante ainda que nenhum campo escape do conjunto de valores esperado.

6. Evidência: mostrando os números por trás da conclusão

Além da explicação em linguagem simples, a tela do tutor exhibe os dados que embasam aquela conclusão:

PetCare AI

⦿ Atenção moderada

Rex está menos ativo e comeu menos que o habitual hoje.

Por que recebi este alerta?

Atividade	Hoje: 39	Média: 75	↓ 48%
Alimentação	Hoje: 58%	Média: 92%	↓ 37%
Score	82	→ 61	

Como esses valores já são calculados pelo Java antes de chamar a IA (seção 5), exibir essa evidência não exige nenhum dado novo — só torna visível o raciocínio que já existe por trás da resposta.

7. Duas capacidades distintas: PetCare AI e PetCare Vision

Pra evitar ambiguidade sobre o que é “inteligência artificial” no produto, o projeto separa nomeação e responsabilidade:

- **PetCare AI** — a camada de LLM + RAG descrita acima: interpreta contexto e explica em linguagem humana.
- **PetCare Vision** — visão computacional clássica (sem LLM) usada no comedouro: compara uma foto do prato antes e depois da refeição e calcula o percentual de ração consumida.

O sensor de peso do comedouro está sendo substituído por uma câmera (ESP32-CAM) para viabilizar essa segunda capacidade. O dado que ela produz (`percentualConsumido`) entra no mesmo fluxo de leituras que já alimenta o Score de Saúde — é uma melhoria de captação de dado, não faz parte da PetCare AI propriamente dita.

Calibração percentual → gramas: o peso em gramas continua sendo registrado (campo `peso_consumido_g`), mas deixa de vir de um sensor de peso e passa a ser calculado no próprio ESP32, a partir do percentual detectado pela câmera e de uma constante de calibração do pote (ex: pote cheio = 40g = 100%). Uma leitura de 50% do prato consumido vira 20g. O ESP32 já envia os dois valores prontos (percentual e gramas) para o Java, então nenhuma mudança é necessária no schema do banco (`LEITURA_COMEDOURO`). Se o pote físico mudar de tamanho no futuro, essa constante de calibração precisa ser atualizada no firmware.

8. Evolução futura (fora do escopo desta sprint)

Duas extensões estão mapeadas para depois que o núcleo estiver estável:

- **Classificação de movimento na borda (TinyML no ESP32):** em vez de enviar apenas variação

- bruta de aceleração, o próprio dispositivo classificaria o movimento (repouso/ativo/agitado) antes de enviar. Fica como evolução por exigir um ciclo próprio de coleta de dado e treinamento, incompatível com o tempo desta sprint.
- **Análise de foto enviada pelo tutor:** o tutor poderia enviar uma foto de algo que notou, e a IA verificaria apenas se há mudança perceptível em relação ao histórico visual do pet — sem nunca descrever ou classificar o que vê, mantendo o mesmo princípio de não diagnosticar. Fica como extensão porque não é bloqueante para as demais funcionalidades do produto.

9. Por que essa não é “só uma chamada de API pra um LLM”

Elemento	Função
Contexto controlado pelo Java	A IA nunca opera sobre dado bruto ou não verificado — o backend decide exatamente o que ela recebe
Urgência calculada por regra determinística	A decisão mais sensível (gravidade) não depende da LLM — vem do Score de Saúde, que é auditável
RAG com base de conhecimento própria	Fundamenta as respostas em diretrizes técnicas reais, reduzindo alucinação
Saída estruturada + validação pós-resposta	Garante que o produto nunca extrapole pra diagnóstico; valores fora do esperado são corrigidos antes de chegar ao tutor
Integração de múltiplas fontes de sensor	A análise combina movimento, alimentação e histórico — não é uma pergunta isolada

10. Repositório e organização

O código-fonte relacionado a esta funcionalidade está organizado no repositório `Disruptive-Architectures` da organização `PetCare-HUB`, com o histórico evolutivo do protótipo IoT (Sprints 1-2) documentado e mantido como base para a captação de dados que alimenta esta funcionalidade.